

Estrutura de Dados

Listas Encadeadas Simples

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Nó: Unidade da Cadeia

Na lista encadeada, cada faixa deixa de ocupar uma posição fixa dentro de um bloco contíguo e passa a viver em um nó independente. Além da faixa, o nó guarda um ponteiro para o próximo nó da cadeia, ou None quando é o último. A estrutura inteira passa a ser sustentada por referências, não por endereços calculados a partir de um índice.



Prof. Everaldo Graciliano Souza Ribeiro

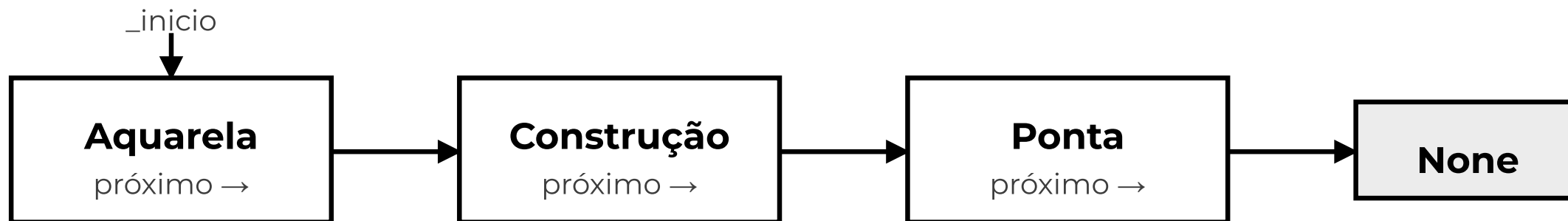
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Anatomia do Nó e a Cadeia até None



cada caixa é um nó: uma faixa + um ponteiro para o próximo

A cadeia termina quando o ponteiro aponta para **None** — não existe posição 4.



Sem Endereço Calculável

O vetor calculava o endereço de qualquer posição com uma única conta. A lista encadeada não tem essa conta: para alcançar a faixa da posição dez é preciso passar pela posição zero, depois pela um, e assim sucessivamente até chegar lá. O acesso direto por índice desaparece — obter uma faixa custa proporcionalmente ao tamanho da playlist.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Inserir Sem Deslocar

Inserir no meio do vetor exigia empurrar todos os elementos seguintes uma casa adiante. Na lista encadeada nada se move: o nó novo aponta para quem viria depois dele, e o nó anterior passa a apontar para o nó novo. Duas atribuições de ponteiro bastam, independentemente de quantos elementos existam antes ou depois da posição afetada.



Prof. Everaldo Graciliano Souza Ribeiro

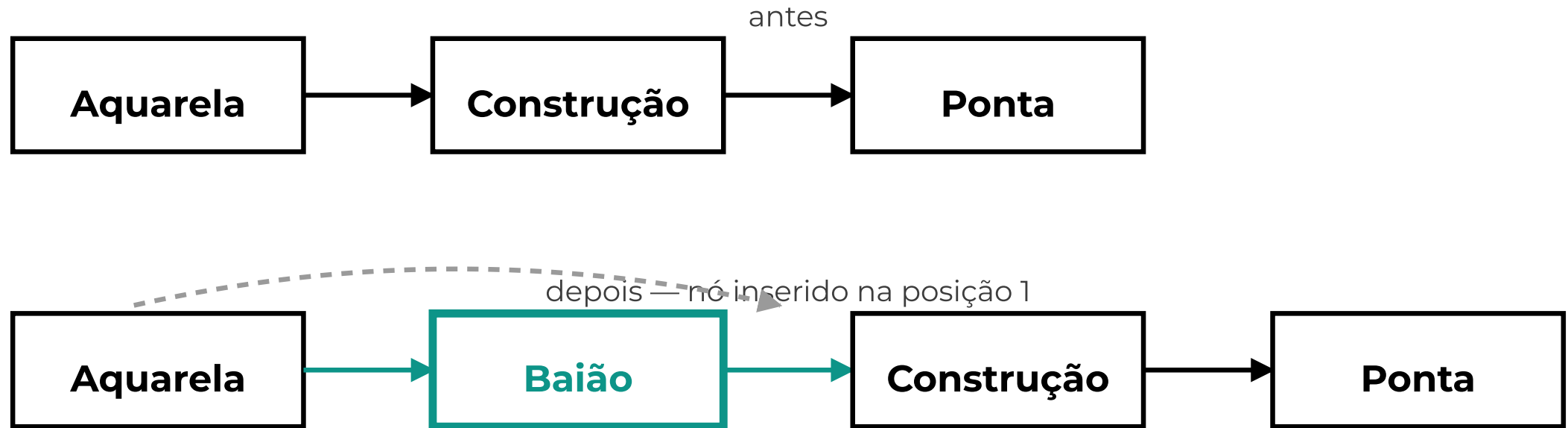
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Inserção no Meio: Antes e Depois



Só **dois** ponteiros são religados; o antigo (arco tracejado) deixa de existir.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Caminhar é $O(n)$, Religar é $O(1)$

Religar dois ponteiros custa sempre o mesmo, não importa onde a inserção ocorra: essa parte é $O(1)$. O custo verdadeiro está em chegar até ali — percorrer a cadeia nó a nó desde o início até a posição anterior, o que é $O(n)$. Inserir no início dispensa esse percurso; inserir perto do fim ainda exige atravessar quase toda a cadeia.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Remover Fecha o Buraco

Remover no vetor deslocava todos os elementos seguintes uma casa para trás, fechando o espaço vazio. Na lista encadeada o nó anterior simplesmente passa a apontar para o nó seguinte ao removido, pulando por cima dele. O nó retirado deixa de ser referenciado por qualquer outro e é descartado automaticamente pelo coletor de lixo do Python.



Prof. Everaldo Graciliano Souza Ribeiro

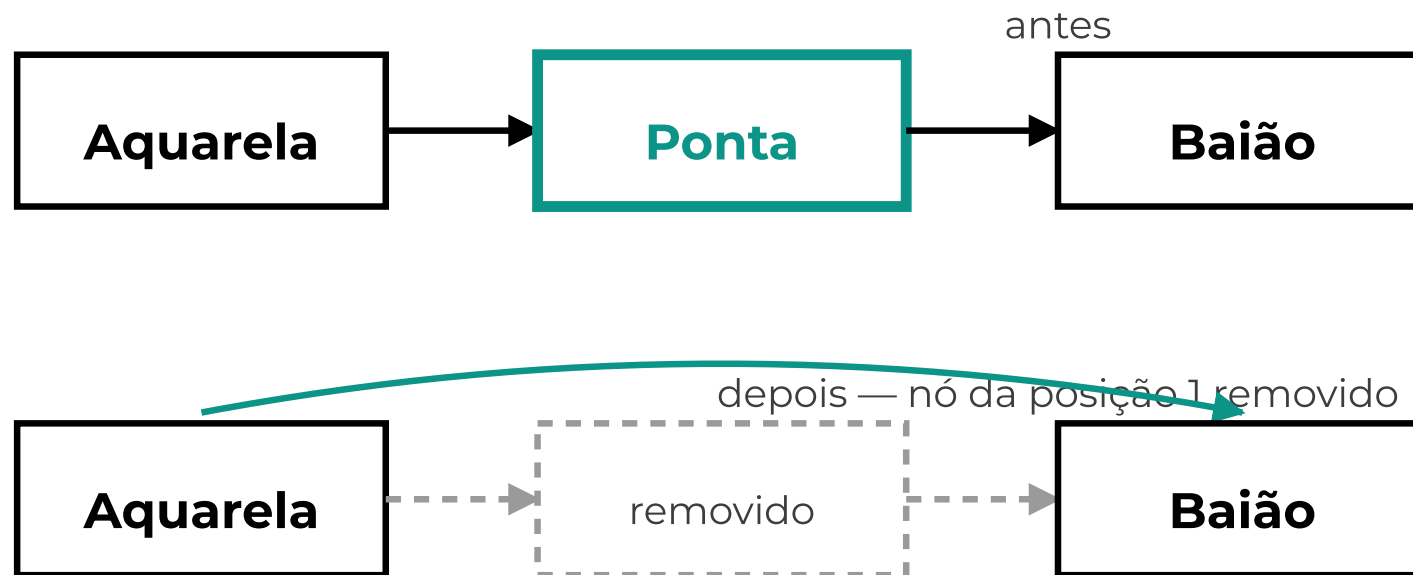
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Remoção no Meio: Antes e Depois



O ponteiro de **Aquarela** pula direto para **Baião**; nada mais aponta para o nó removido.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Quando a Lista Encadeada Compensa

A lista encadeada compensa quando inserções e remoções em posições conhecidas são frequentes e o acesso por índice é raro. Perde quando o programa precisa alcançar posições arbitrárias com frequência, porque cada obtenção exige percorrer a cadeia desde o início. A escolha entre vetor e lista encadeada depende de qual operação domina o uso real da estrutura.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Nó e o Construtor da Playlist

```
class _No:
    """Um elo da cadeia: guarda uma faixa e aponta para o proximo no."""

    def __init__(self, faixa):
        self.faixa = faixa
        self.proximo = None

class Playlist:
    def __init__(self, nome):
        self.nome = nome
        self._inicio = None
        self._fim = None
        self._tamanho = 0
```



Adicionar: o Ponteiro `_fim` Evita Percorrer

```
def adicionar(self, faixa):  
    """Acrescenta no fim. Custo constante: o ponteiro  
    _fim evita percorrer a cadeia inteira."""  
    novo = _No(faixa)  
    if self._fim is None:  
        self._inicio = novo  
    else:  
        self._fim.proximo = novo  
    self._fim = novo  
    self._tamanho += 1
```



Inserir: Caminhar e Depois Religar

```
def inserir_em(self, posicao, faixa):
    if not 0 ≤ posicao ≤ self._tamanho:
        raise IndexError("posicao fora da playlist")
    anterior = self._no_em(posicao - 1) if posicao > 0 else None
    novo = _No(faixa)
    if anterior is None:
        novo.proximo = self._inicio
        self._inicio = novo
    else:
        novo.proximo = anterior.proximo
        anterior.proximo = novo
    if novo.proximo is None:
        self._fim = novo
    self._tamanho += 1
```



Remover: Religar Para Fechar o Buraco

```
def remover_em(self, posicao):  
    if not 0 ≤ posicao < self._tamanho:  
        raise IndexError("posicao fora da playlist")  
    anterior = self._no_em(posicao - 1) if posicao > 0 else None  
    if anterior is None:  
        removido = self._inicio  
        self._inicio = removido.proximo  
    else:  
        removido = anterior.proximo  
        anterior.proximo = removido.proximo  
    if removido is self._fim:  
        self._fim = anterior  
    self._tamanho -= 1  
    return removido.faixa
```



Atividades

Exercício 1

Numa playlist de 100 faixas encadeadas, quantos nós são percorridos para inserir na posição 0? E na posição 99?

Exercício 2

Por que obter(50) agora custa mais caro que obter(0), ao contrário do que ocorria no vetor?

Exercício 3

Depois de um nó ser removido com `remove_em`, por que ele desaparece da memória sem nenhum comando explícito para apagá-lo?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Criar `contar_nos()` na Playlist, percorrendo a cadeia sem usar `_tamanho`, só para comparar o resultado.
2. Medir e exibir quantos nós `_no_em` percorreu na última operação da sessão.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>